Design Slack

SystemsExpert design walkthrough

Clarifying questions

- Channels and DMs? Threads, reactions, file uploads?
- Max workspace size?
- Must messages sync across devices with unread counts?

Functional requirements

- Send/receive messages in channels and DMs in real time
- Load message history with pagination
- Unread indicators and mentions
- Workspaces/organisations

Non-functional requirements

- Messages delivered in < ~500 ms
- No message loss; ordering per channel
- Large orgs (100k members) supported

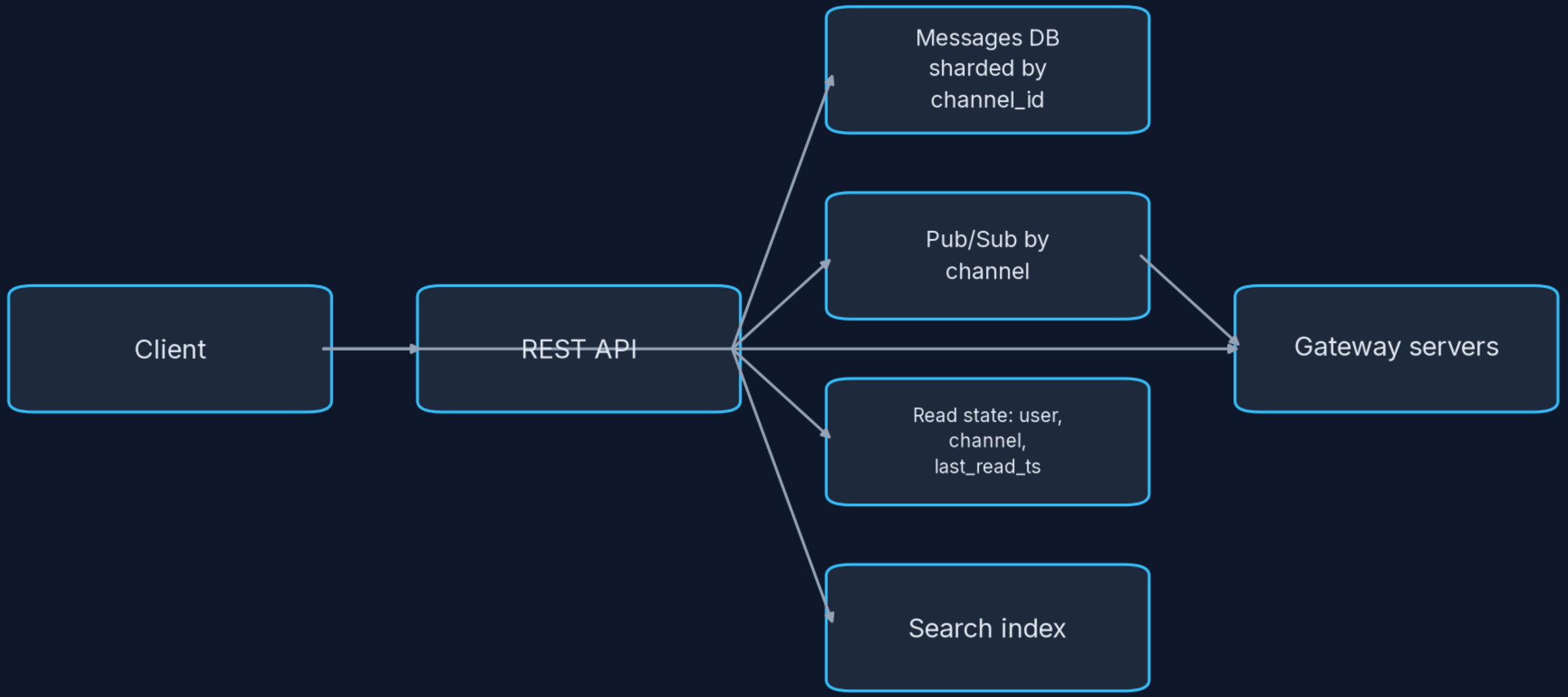
Capacity estimates

- ~10M DAU, ~100 messages/user/day → ~1B messages/day ~ 12k/s avg
- Messages ~1 KB → ~1 TB/day

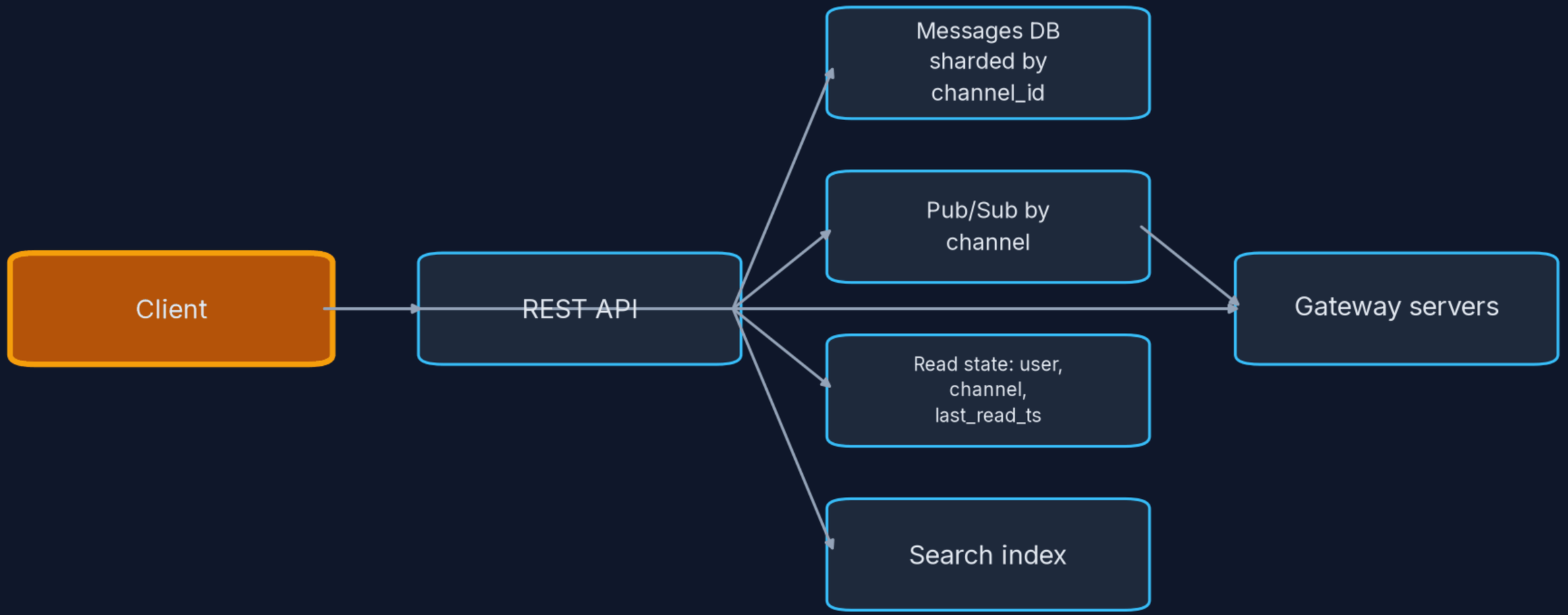
Capacity: worked out

- 10M DAU × 100 msgs = 1B messages/day
- 1B ÷ 86,400 ≈ 12k msgs/s (peak ~3× ≈ 35k)
- × 1 KB ≈ 1 TB/day ≈ 365 TB/year
- Fan-out to ~10 online members ≈ 350k pushes/s

High-level architecture

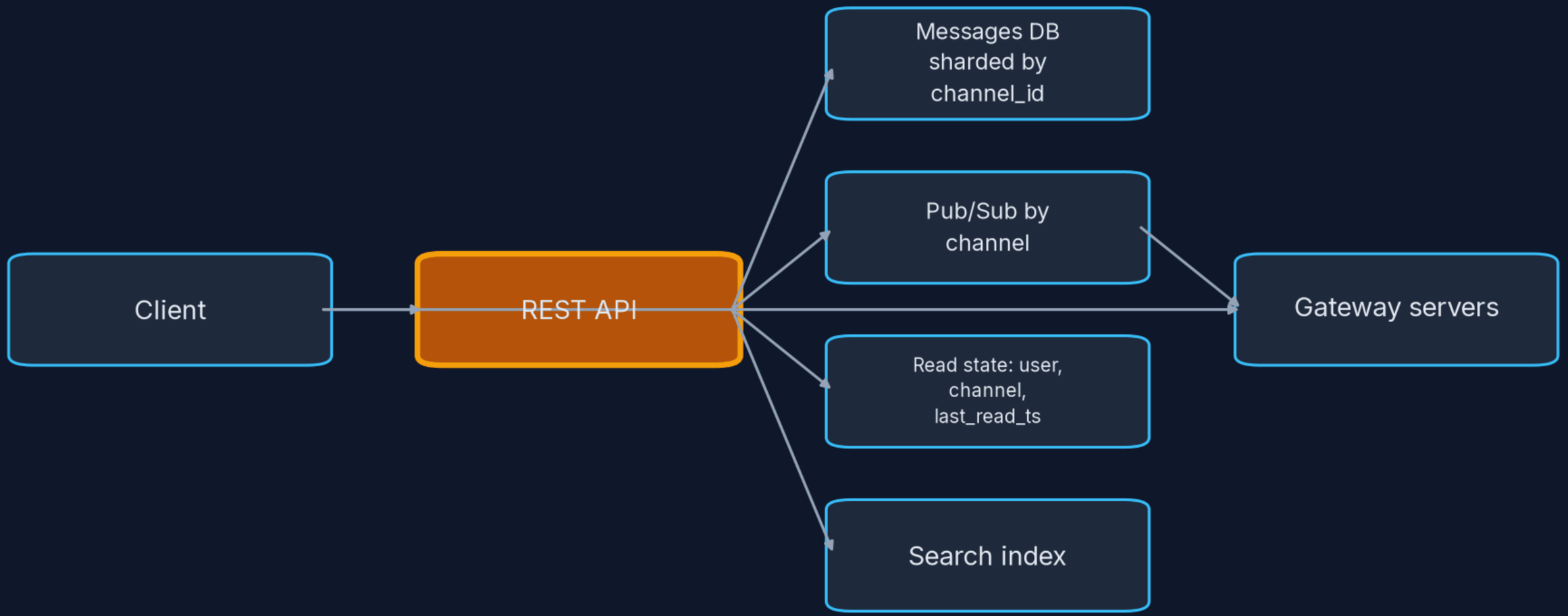


Request flow: Send a message



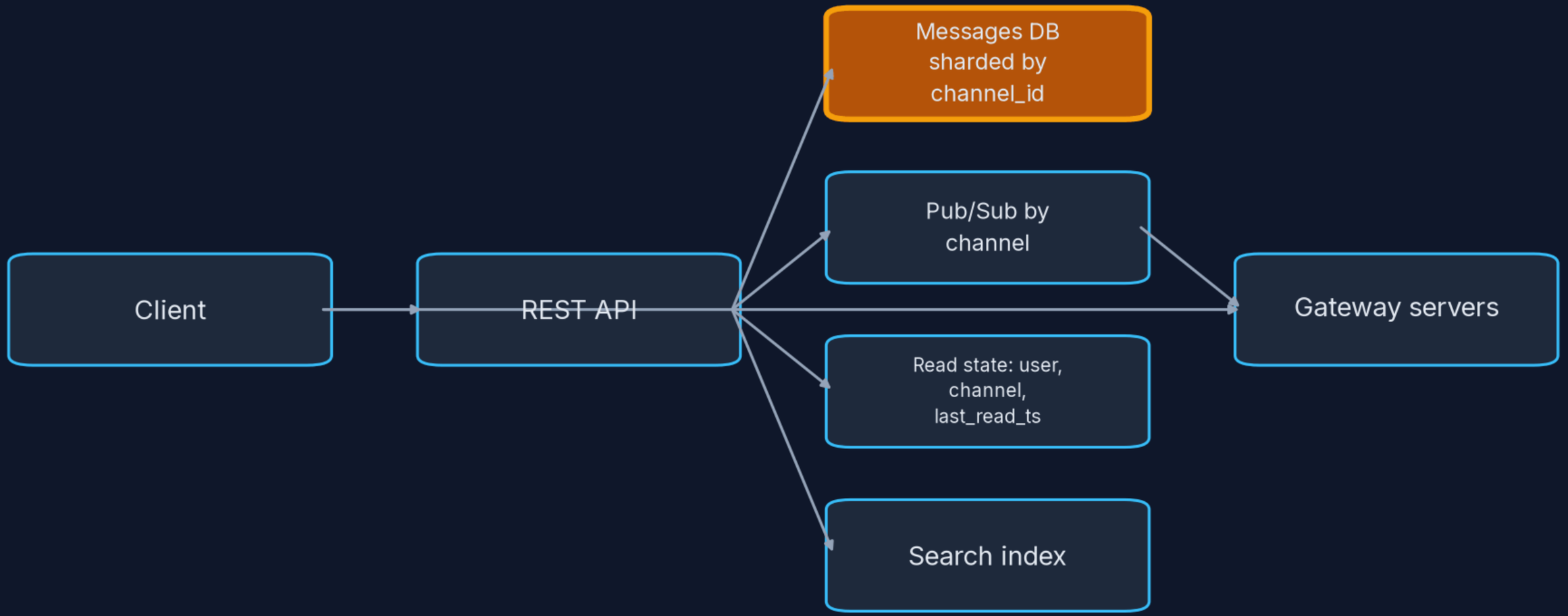
1. User sends a message

Request flow: Send a message



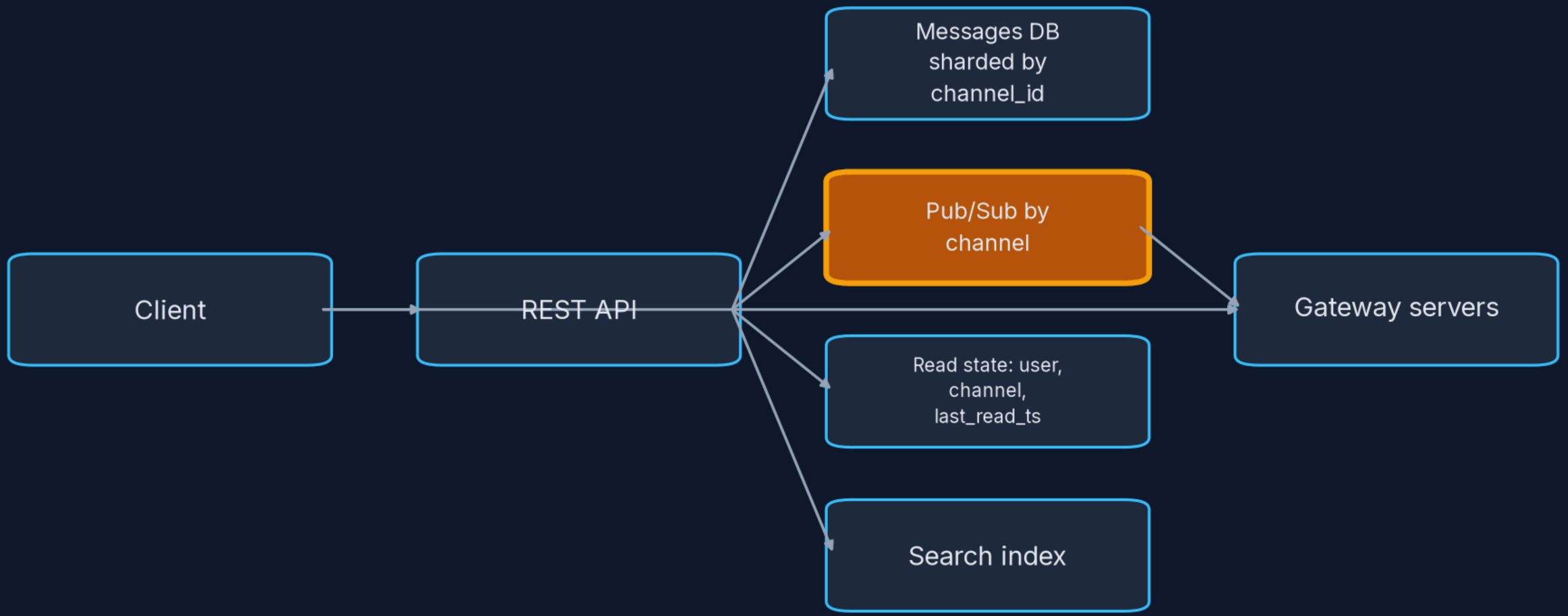
2. API persists it

Request flow: Send a message



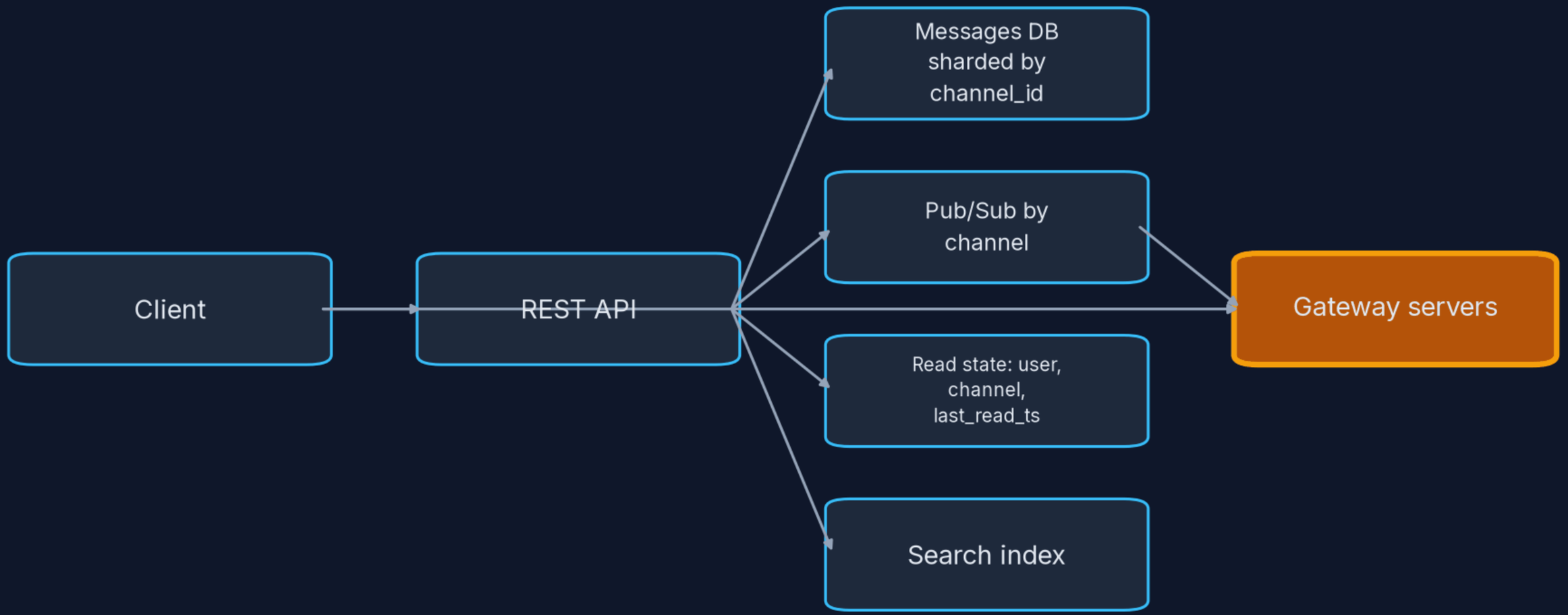
3. Stored in the channel shard

Request flow: Send a message



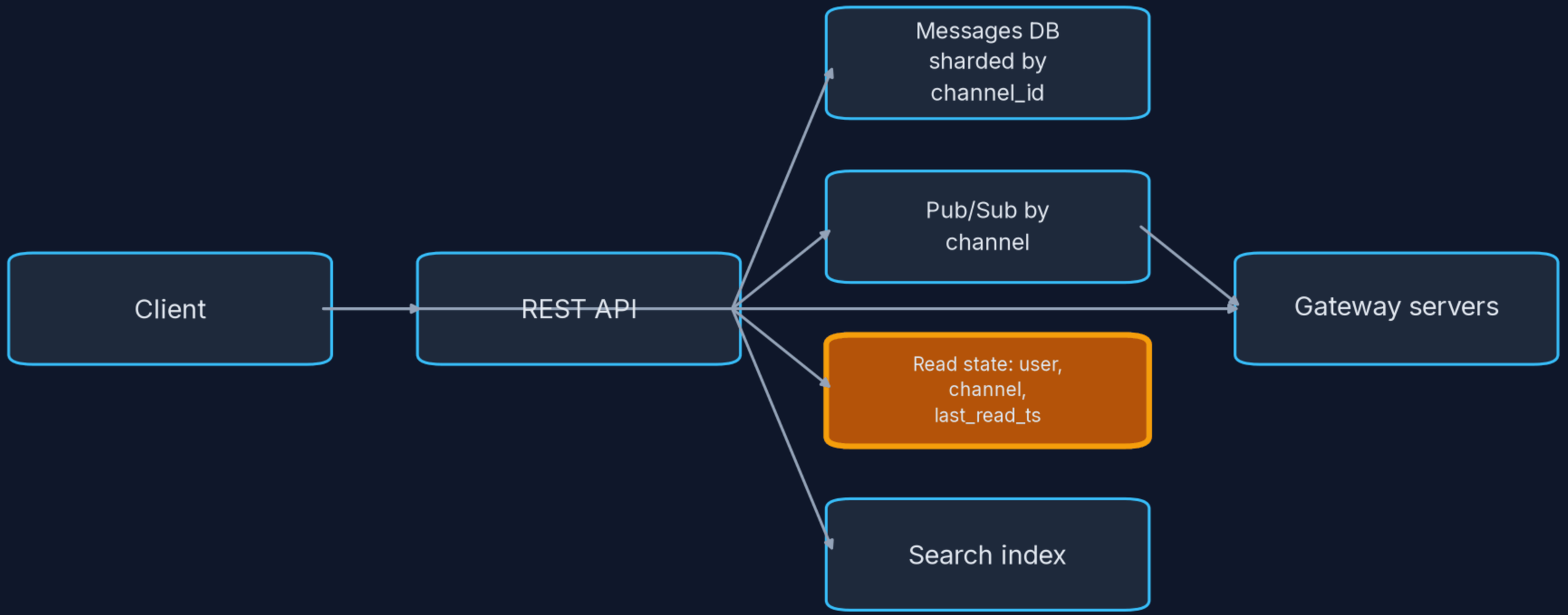
4. Published to the channel topic

Request flow: Send a message



5. Gateways push to members

Request flow: Send a message



6. Unread counts update

Data model

- Channels(id, workspace_id, name, type)
- Memberships(channel_id, user_id)
- Messages(channel_id, ts, author_id, body, thread_ts)
  PK(channel_id, ts)
- Read_state(user_id, channel_id, last_read_ts)

API

```
POST /channels/{id}/messages {text, clientMsgId}
GET  /channels/{id}/messages?before=ts&limit=50
POST /channels/{id}/read {ts}
WS   events: message.new, message.edited, reaction.added, typing
```

Real-time fan-out

- Gateways subscribe to channels their connected users belong to; a message is persisted first, then published; gateways push to sockets.
- Very large channels fan out through hierarchical pub/sub.

Ordering

- Server-assigned ts per channel (monotonic per shard); client dedupes with clientMsgId.

Unreads

- Compare channel latest ts with read_state; computed on connect and kept live by events.

Key decisions

- Shard messages by channel → channel reads stay on one shard
- WebSocket gateways + pub/sub → real-time delivery
- Server timestamps per channel → consistent ordering
- Client message IDs → dedupe on retry

Trade-offs & alternatives

- Sharding by channel keeps history queries local; huge workspaces need channel-level partitioning
- Persist-then-publish adds a few ms but guarantees no lost messages

Recap

- Clarify scope, then functional + non-functional needs
- Estimate load to size storage and fan-out
- Core pieces: Client, Gateway servers, REST API, Messages DB sharded by channel_id, Pub/Sub by channel
- Know your trade-offs and failure modes